Lab 7

Rudi Aleidan

Task 1: using models, we create a database scheme

To have the scheme in the database, I wrote the following in `apps/bookmodule/models.py`:

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length = 50)
    author = models.CharField(max_length = 50)
    price = models.FloatField(default = 0.0)
    edition = models.SmallIntegerField(default = 1)
```

Then, I ran the following commands in the terminal to create the database scheme:

```
python manage.py makemigrations
python manage.py migrate
```

Screenshot of the terminal output:

```
(djangoenv) Rudi libraryproject ❯ python3 manage.py makemigrations
Migrations for 'bookmodule':
  apps/bookmodule/migrations/0001_initial.py
    + Create model Book
(djangoenv) Rudi libraryproject ❯ python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bookmodule, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying bookmodule.0001_initial... OK
  Applying sessions.0001_initial... OK
```

Task 2: using the extension `SQLite3 Editor` to view the database and insert fake data into it

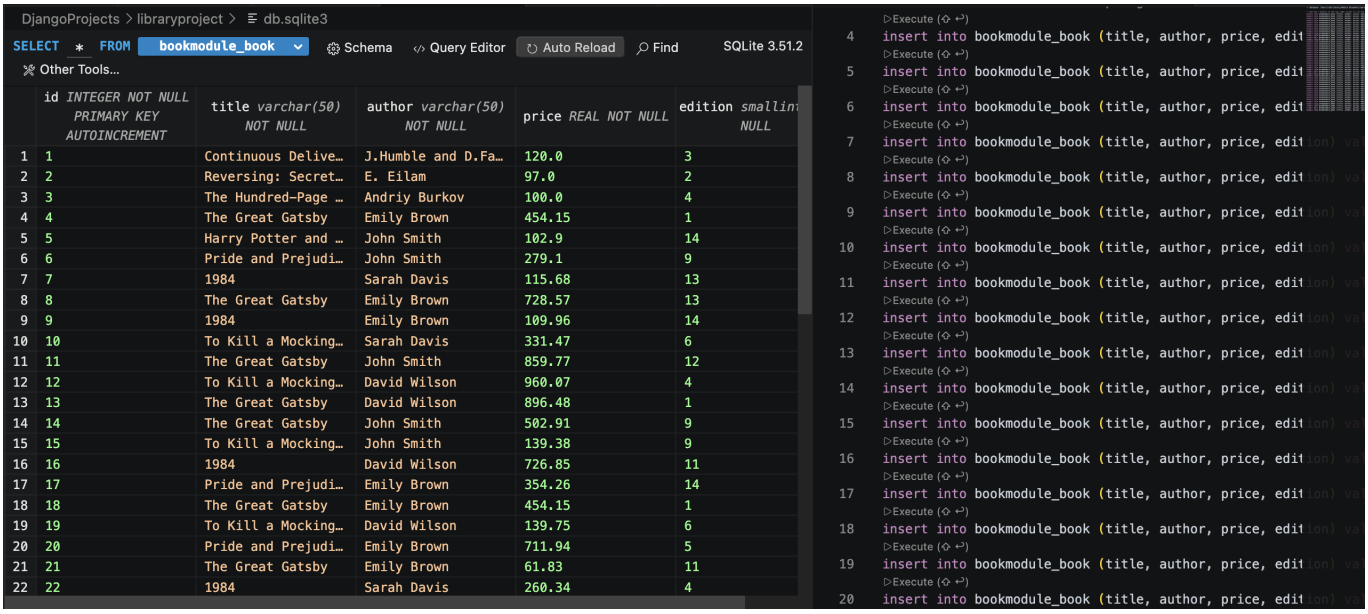
After opening the database file `db.sqlite3` with the extension `SQLite3 Editor`, the following table weas found:

DjangoProjects > libraryproject > db.sqlite3

SELECT * FROM bookmodule_book | Schema | Query Editor | Auto Reload | Find | Other Tools...

id *INTEGER NOT NULL PRIMARY KEY*	title *varchar(50) NOT NULL*	author *varchar(50) NOT NULL*	price *REAL NOT NULL*	edition *smallint NOT NULL*

Then, I generated some fake data using the website `https://www.mockaroo.com/` and inserted it into the table:



Task 3: Use simple queries using Django ORM

To query the database, I wrote the following code in `apps/bookmodule/views.py`:

```
from .models import Book

# ...

def simple_query(request):
    mybooks = Book.objects.filter(title__icontains='and') # <- multiple objects
    return render(request, 'bookmodule/bookList.html', {'books':mybooks})
```

Then, I added the following path to `apps/bookmodule/urls.py`:

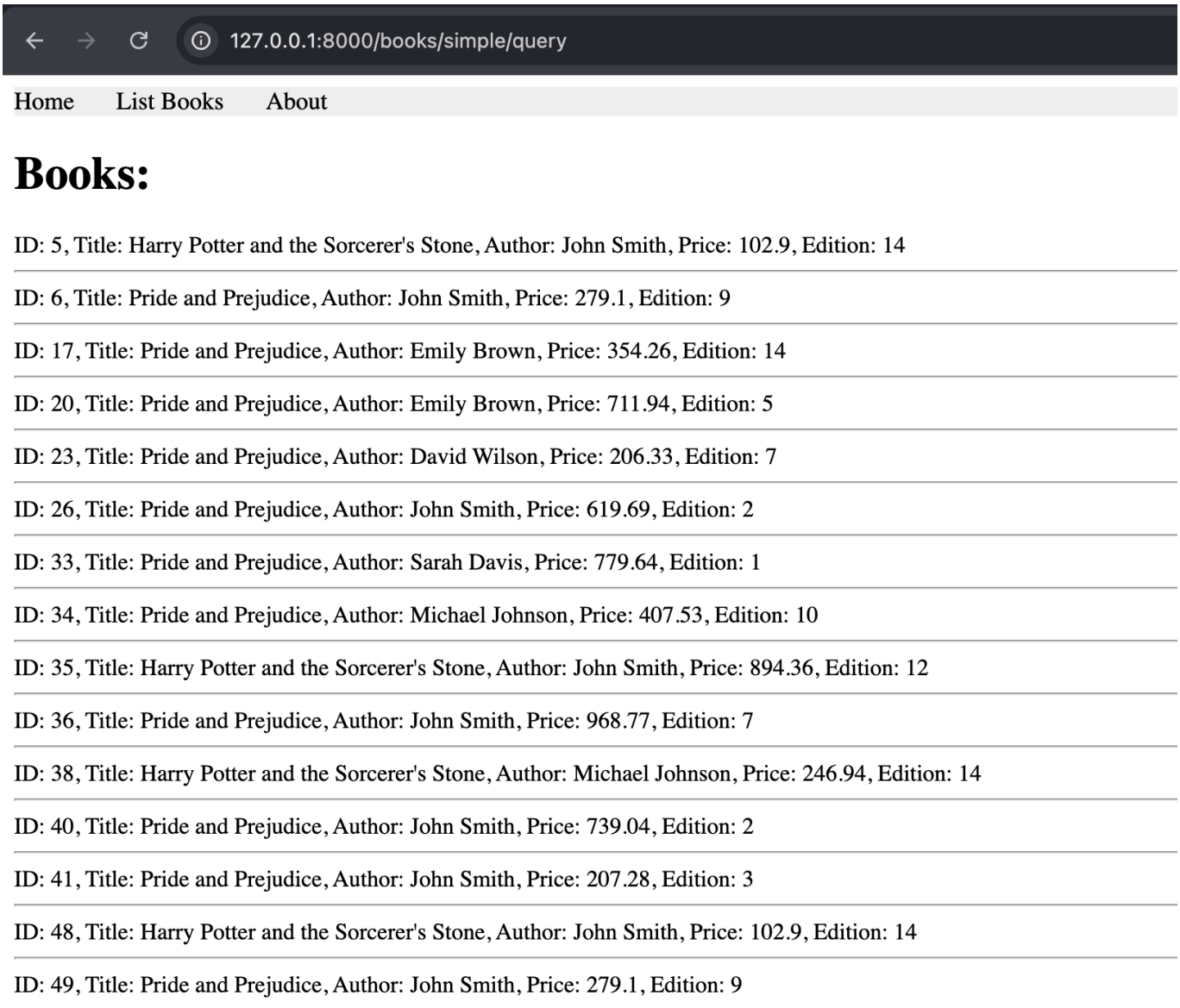
```
path('simple/query', views.simple_query, name="books.simple_query"),
```

In `bookmodule/bookList.html`, added Django tag to list books:

```
{% extends "layouts/base.html" %}
{% load static %}
{% block title %} Books {% endblock title %}
{% block content %}
<h1>Books:</h1>
```

```
  {% for obj in books %}
  ID: {{ obj.id }}, Title: {{ obj.title }}, Author: {{ obj.author }},
Price: {{ obj.price }}, Edition: {{ obj.edition }}
    <hr>
  {% endfor %}
{% endblock content %}
```

Screenshot at `http://127.0.0.1:8000/books/simple/query`, it shows the books whose title contains the string "and":



Task 4: Customize queries using common lookup conditions and keywords

In `apps/bookmodule/views.py`, I added the following code to query the database with multiple conditions:

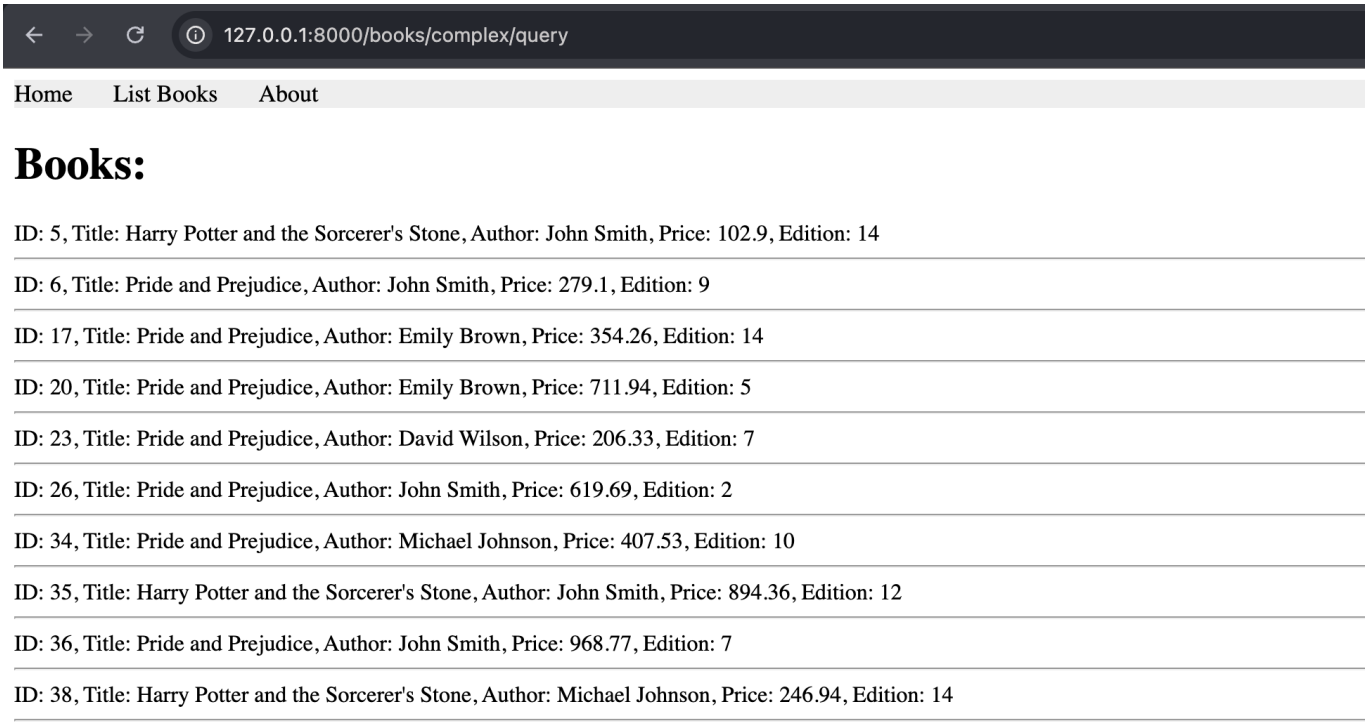
```
def complex_query(request):
    mybooks =
```

```
Book.objects.filter(author__isnull=False).filter(title__icontains='and').f
ilter(edition__gte=2).exclude(price__lte=100)[:10]
    if len(mybooks) >= 1:
        return render(request, 'bookmodule/bookList.html', {'books':
mybooks})
    else:
        return render(request, 'bookmodule/index.html')
```

Conditions are:

- `author__isnull=False` — the book must have an author.
- `title__icontains='and'` — the title must contain the word "and".
- `edition__gte=2` — the edition must be 2 or higher.
- `exclude(price__lte=100)` — price must be above 100.
- `[:10]` — only return first 10 results. If at least one book matches, it displays them in `bookList.html`. Otherwise, it redirects to `index.html`.

Then, I added the following path to `apps/bookmodule/urls.py`:

```
path('complex/query', views.complex_query, name="books.complex_query"),
```

Screenshot at `http://127.0.0.1:8000/books/complex/query`: